

■ Design Online Auction System (eBay)

System Design Interview | E-Commerce & Marketplace

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional

List items with start price & end time; Place bids; Auto-bidding (proxy bid up to max); Real-time bid updates; Winner determination; Payment

Non-Functional

Bid acceptance <200ms; No two winners; Real-time bid display to all viewers; Handle auction-end spike; Audit trail for all bids

Scale

eBay: 1.5B listings; Popular auctions: thousands bidding in final seconds ('sniping'); must handle last-second bid storm

■ High-Level Architecture

- Listing Service (CRUD for items), Bid Service (bid placement + validation), Auction Engine (determines winner), Notification Service
- Real-time bid feed: WebSocket connections to Bid Service; new bid published to Kafka → fanned out to all viewers of that auction
- Auction state machine: DRAFT → ACTIVE → ENDING_SOON (last 5 min) → CLOSED → SETTLED
- Winner determination: job triggered at auction end time; reads highest valid bid from DB; initiates payment flow

■ Bid Processing — Concurrency is Critical

- **Optimistic Locking:** auctions table has current_price + version; UPDATE SET current_price=new_bid, version=v+1, leading_bidder=user WHERE id=X AND version=v AND current_price < new_bid; retry on conflict
- **Redis for Speed:** Cache current_price and leading_bidder in Redis; validate bid against Redis (fast); persist to DB async; display from Redis
- **Bid Validation:** Bid must be > current_price + min_increment; user must have verified payment method; item must be ACTIVE status
- **Proxy Bidding:** Store user's max_bid secretly; when outbid, auto-increment up to max_bid; only reveal minimum needed to lead
- **Audit Trail:** Every bid attempt (success or fail) appended to bids table; immutable; used for dispute resolution

■ Data Model

- auctions: auction_id, seller_id, title, description, start_price, reserve_price, current_price, leading_bidder_id, end_time, status, version
- bids: bid_id, auction_id, bidder_id, amount, max_proxy_amount, timestamp, status(WINNING/OUTBID/REJECTED) — append only
- Index: (auction_id, amount DESC) for winner lookup; (bidder_id) for user bid history; (end_time, status) for scheduler

- Notifications: subscribe model — users watching auction stored in Redis SET; fan-out bid updates to set members

■ Auction-End Handling

- Scheduler: dedicated job scheduler checks auctions ending in next 60s every 10s; processes end atomically
- Last-second bids: extend auction by 5 min if bid placed in last 2 min (like eBay's anti-sniping); prevents bid sniping
- Winner notification: async via Kafka → Notification Service → email + push; payment initiated automatically
- No-sale: if final price < reserve_price → notify seller and highest bidder; no obligation to sell

■ Bid Storage Strategy

Sync DB Write	Redis-First + Async DB
✓ Strong consistency	✓ Fast response (<10ms)
✓ Simpler conflict detection	✓ Redis atomic ops
✓ Slower (150-300ms)	✓ Async DB persistence
✓ DB bottleneck under storm	✓ Risk if Redis fails before flush

■ Fraud Prevention

- Shill bidding: detect seller using alternate accounts to inflate price; flag if bidder IP/device matches seller
- Bid retraction limits: eBay allows retractions only in specific circumstances; tracked per user with cooling-off period
- Verified payment method required before bidding; hold authorization on payment method for auction duration
- Rate limiting: max N bids per minute per user per auction; bot detection via behavioral analysis